Diseño de la Arquitectura

La solución se construyó sobre Kubernetes para orquestrar y administrar todos los servicios describen en este proyecto.

Con el fin de organizar mejor la infraestructura y separar responsabilidades, se definieron dos Namespaces dentro del clúster:

- publico
- privado

Recursos en el Namespace Público

Dentro de este namespace se ubican los componentes que deben ofrecer acceso externo, ya que actúan como intermediarios entre los clientes y los servicios internos.

Router

El Router se implementa a partir de una imagen base de CentOS y se configura utilizando nftables.
Su función principal es redirigir el tráfico entrante hacia los distintos servicios internos. En este caso, el tráfico TCP y UDP en los puertos definidos se dirige a FreeSWITCH, mientras que las peticiones al puerto TCP/8080 son redirigidas a NGINX 1, las del puerto TCP/8081 hacia NGINX 2, y finalmente, el puerto TCP/80 se encamina al Ingress Controller.

Ingress Controller

El Ingress Controller se implementa utilizando Apache Server y tiene como responsabilidad redirigir las peticiones HTTP en función de la URL solicitada.
En esta configuración, cuando un cliente accede a `http://127.0.0.1:80/Nginx1`, la solicitud se dirige hacia NGINX 1, mientras que si accede a `http://127.0.0.1:80/Nginx2`, la petición es redirigida hacia NGINX 2.

Recursos en el Namespace Privado

En este namespace se alojan los servicios internos que no deben ser accesibles directamente desde el exterior, garantizando así aislamiento y seguridad dentro del clúster.

NGINX 1

NGINX 1 es un servidor web simple que, al ser accedido, muestra el mensaje `"Hola NGINX1"`.
Para su implementación en Kubernetes se realizaron los siguientes pasos:

- Se creó un Deployment con una réplica del pod que corre NGINX 1, utilizando la imagen nginx:alpine.
- Se definió un Service de tipo ClusterIP para permitir la comunicación interna dentro del clúster.
- Se creó un ConfigMap para establecer la configuración del HTML que se servirá.
- Este ConfigMap se montó como un volumen dentro del Deployment, de modo que el contenedor pueda leer y servir el contenido correcto del sitio web.

NGINX 2

NGINX 2 tiene la misma estructura y propósito que NGINX 1, mostrando el mensaje `"Hola NGINX2"`.
Su implementación también incluyó:

- Deployment con una réplica, utilizando la imagen nginx:alpine.
- Service de tipo ClusterIP.
- ConfigMap para la configuración del HTML.
- Montaje del ConfigMap como volumen en el Deployment, garantizando que el contenedor pueda servir la página correctamente.

FreeSWITCH

FreeSWITCH es un servicio de telefonía IP que utiliza el protocolo SIP (Session Initiation Protocol) sobre los transportes UDP y TCP. Este servicio permite crear extensiones telefónicas, realizar llamadas entre ellas y simular un entorno de central telefónica dentro del clúster.

Para su implementación en Kubernetes se realizaron los siguientes pasos:

- Se creó un Deployment utilizando la imagen safarov/freeswitch.
- Se definió un ConfigMap que contiene la configuración necesaria para los usuarios en el archivo `users.xml`, el `dialplan.xml` para los sonidos y el archivo `event_socket.conf.xml` para que freeswitch escuche en todas las interfaces de red disponibles.
- Este ConfigMap se montó como un volumen dentro del Deployment, de manera que el contenedor pueda cargar la configuración correctamente al iniciar.
- Se creó un Service de tipo ClusterIP para exponer FreeSWITCH dentro del clúster y pueda comunicarse con otros pods dentro del cluster.

El servicio de FreeSWITCH escucha en el puerto 5060 tanto en UDP como en TCP, permitiendo así el funcionamiento del protocolo SIP para llamadas y comunicaciones entre extensiones.

Herramientas

Para implementar los servicios se utilizó Helm, una herramienta que permite definir la infraestructura como código.
Esto no solo simplifica el despliegue en Kubernetes, sino que también asegura que la configuración sea consistente, fácil de mantener y reutilizable en distintos entornos.

Instalación

Para desplegar todos los servicios del proyecto en Kubernetes, simplemente se ejecuta el siguiente comando de Helm:

```
helm install app app
```

Con este comando se crea automáticamente toda la infraestructura definida en el chart, incluyendo Deployments, Services, ConfigMaps y demás recursos necesarios. Una vez finalizado, los servicios quedarán listos para ser utilizados sin necesidad de configuraciones adicionales.

Pruebas Funcionales

Para verificar el correcto funcionamiento de los distintos servicios del proyecto, se llevaron acabo diversas pruebas funcionales.

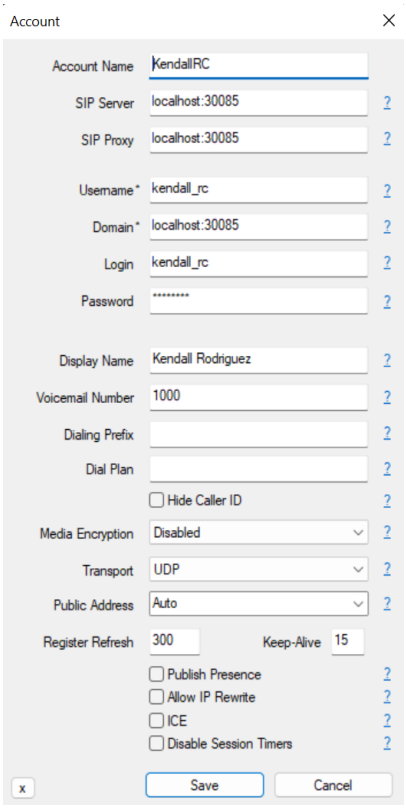
FreeSwitch

Para verificar el correcto funcionamiento del servidor FreeSWITCH, se utilizó un cliente SIP, llamado MicroSIP, que permitió conectarse al servidor y realizar llamadas exitosas entre los usuarios registrados.

A continuación se muestran algunas imagenes utilizando MicroSIP donde se demuestra la conexión exitosa con el servidor

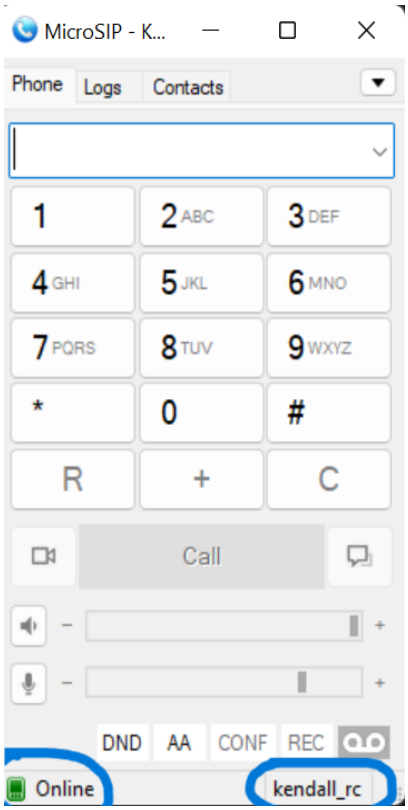
1. Ajustes y configuración del cliente

Se establecen la configuración del servidor y el usuario ya previamente registrado. En la sección inferior se indica el tipo de protocolo, en este caso se estableción UDP, sin embargo, funciona de igual manera con TCP, ambos protocolos escuchan en el mismo host y puerto



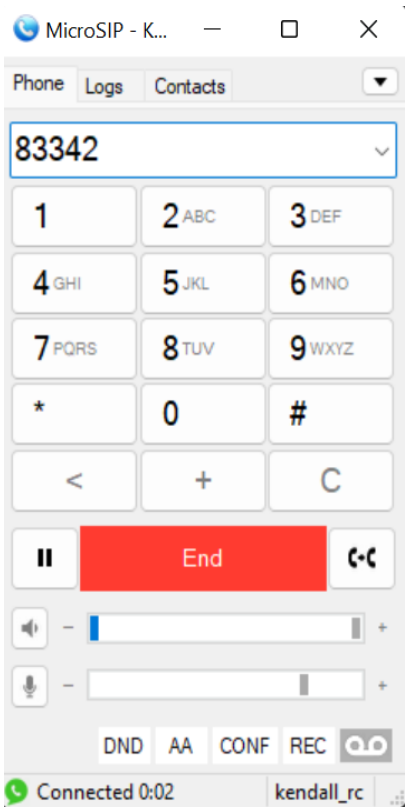
2. Conexión establecida

La conexión se establece exitosamente, y el usuario queda registrado y listo para comunicarse a través del servidor. Esto confirma que tanto el cliente SIP como el servidor FreeSWITCH funcionan correctamente, permitiendo iniciar llamadas y transmitir información de manera confiable entre los usuarios.



3. Llamada a otro usuario

Se establece la conexión con el número 83342, previamente registrado en el servidor, utilizando el cliente SIP. Una vez iniciada la llamada, el sistema muestra en la parte inferior la duración de la comunicación con el usuario, que en este caso es de 2 segundos.



Conclusiones

- FreeSWITCH se implementó exitosamente. Las pruebas funcionales demostraron que el servidor gestiona correctamente las llamadas entre los usuarios. Además, se verificó que los usuarios pueden registrarse en el servidor sin inconvenientes, que el enrutamiento de las llamadas se realiza de forma adecuada y que tanto los protocolos UDP como TCP funcionan correctamente en el mismo host y puerto. Esto confirma que la configuración del servidor y de los clientes SIP es estable y cumple con los objetivos del proyecto.

Recursos de Apoyo

- Documentación oficial de FreeSWITCH
- Helm - sitio oficial
- Kubernetes - documentación oficial